Twins as Processes: A Pure Erlang/OTP Framework for Digital Twins Across the Edge–Cloud Continuum

[Author Name]
School of Computing Science
Simon Fraser University
Burnaby, BC, Canada
[email]

***Abstract*—The digital twin has matured from a NASA-era spacecraft analogue into a foundational construct of Industry 4.0, yet its conceptual and architectural fragmentation persists: the term spans one-way digital shadows, full closed-loop cyber-physical systems, and cognitive agents, with no agreement on the runtime substrate that should host them. Drawing on the design space codified in the *Handbook of Digital Twins*, this paper argues that the recurring requirements a twin framework must satisfy—per-asset autonomy, hierarchical composition from component to system-of-systems, variable synchronization, fault containment, live model evolution, and deployment that ranges from an embedded device to a cloud cluster—are precisely the properties that the actor model provides, and that the Erlang/Open Telecom Platform (OTP) runtime, the BEAM, realizes them more directly than either vendor twin platforms or actor frameworks ported onto general-purpose runtimes. We develop the correspondence in detail, mapping each axis of the Handbook taxonomy onto a native BEAM mechanism, and from it derive an actor-oriented framework in pure Erlang. A twin is a supervised `gen_statem` process that carries an explicit split between observed and modelled state, whose divergence operationalizes the digital-shadow-to-digital-twin maturity ladder; asset topology is expressed as a supervision hierarchy of composite twins; transport is isolated behind adapters; and registry, history, and time are pluggable behaviours. The same twin module runs as a thin leaf under AtomVM on a microcontroller and as a composite in a distributed cluster, with live code replacement updating a running model without halting the asset it mirrors. We ground the design in a water-distribution case study built on the C-Town benchmark and a simulator-replay adapter, and discuss the limits of the approach, notably numerical modelling and ecosystem maturity.**

***Keywords*—digital twin, actor model, Erlang, BEAM, OTP, edge computing, cyber-physical systems, supervision, Industry 4.0, AtomVM**

I. Introduction

The digital twin (DT) has become one of the organizing ideas of digital transformation, promising a virtual counterpart that is bound to a physical asset closely enough to monitor, predict, and ultimately steer it throughout its lifecycle [1], [2], [3]. The breadth of that promise is also the source of a persistent difficulty. As Agrawal and Fischer observe in their conceptual treatment, practitioners routinely disagree on what a twin even is: some take it to mean a complex, predictively capable real-time model, others a lightweight digital representation, and the boundary between a twin and the adjacent vocabulary of Building Information Modelling, cyber-physical systems, and Industry 4.0 is blurry in practice [4], [3]. This ambiguity is not merely terminological. It propagates into architecture, where it becomes a question of what software substrate should host a twin, and the literature answers that question far less often than it answers the question of what a twin is for.

Two families of answer dominate. The first is the integrated platform: managed cloud services such as Azure Digital Twins and open frameworks such as Eclipse Ditto provide a graph of twin objects, an ingestion pipeline, and a query surface, but they assume a cloud-centric deployment and treat the twin as a document or a node in a managed graph rather than as a live computational entity [29], [28]. The second is the actor framework, in which each twin is modelled as an actor—an independent unit of state and behaviour that communicates only by messages. This lineage is the more faithful one, because a population of physical assets, each evolving on its own and interacting through well-defined couplings, is almost a textbook instance of the actor model of Hewitt and of Agha [5], [6]. Microsoft's Orleans and its virtual-actor abstraction made this practical at cloud scale, and its ideas have been carried onto the BEAM in the form of Erleans [7], [8]. Yet the actor framework is usually retrofitted onto a runtime—the CLR, the JVM—whose scheduler, memory model, and failure semantics were designed for something else.

This paper takes the actor reading seriously and follows it to its conclusion. If a twin is most naturally an actor, then the right substrate is the runtime for which the actor model is not a library but the native execution model. That runtime is the BEAM, the virtual machine underlying Erlang and the Open Telecom Platform [9], [10]. Our central claim is that the design space the *Handbook of Digital Twins* enumerates—the levels at which a twin can sit, the maturity ladder from mirror through shadow to twin, the topology binding twin to asset, the synchronization regime, the decision loop, and the classes of user a twin must serve—is not an inconvenient list of features to be implemented on top of a substrate but a set of requirements that OTP already satisfies through processes,

supervision trees, transparent message passing, distribution, and hot code loading [1], [11], [12]. Building a DT framework in pure Erlang is therefore less an act of construction than one of naming: most of the hard parts are already mechanisms in the runtime.

We make three contributions. First, we read the Handbook's vendor-neutral taxonomy as a specification and establish a point-by-point correspondence between its axes and BEAM mechanisms, arguing that the fit is structural rather than incidental. Second, from that correspondence we derive an actor-oriented framework whose core abstraction is a twin as a supervised state machine carrying an explicit observed-versus-modelled state split, with asset topology expressed directly as a supervision hierarchy and with transport, addressing, history, and time isolated behind small behaviours. Third, we identify the edge-to-cloud span as the property that distinguishes a BEAM-native framework from both platforms and ported actor frameworks: the same twin module runs on a microcontroller under AtomVM and in a cloud cluster with identical semantics, and a running model can be replaced in place. We ground the design in a water-distribution case study and are candid about where the approach is weak.

The remainder of the paper is organized as follows. Section II reconstructs the DT design space from the Handbook. Section III argues for the actor model and for the BEAM specifically. Section IV presents the framework architecture. Section V develops the edge-to-cloud argument. Section VI describes the water-distribution case study. Section VII situates the work, and Section VIII discusses limitations and future directions before Section IX concludes.

II. The Digital-Twin Design Space

To argue that a runtime fits the DT problem, one must first fix what the problem is. We reconstruct it from the Handbook, which is useful precisely because it is a consensus document assembled from more than a hundred contributors rather than the position of a single vendor or school [1].

II-A. From the Apollo Twin to the Mirrored Spaces Model

The genealogy of the idea is instructive. During the Apollo programme NASA maintained physical duplicates of its spacecraft on the ground, using them for training and for rehearsing responses to in-flight contingencies; every high-fidelity prototype that reproduces operating conditions is in this sense a twin [4]. Because building a physical duplicate of every asset is impractical, the twinning idea was moved into software. Grieves introduced the notion of virtually twinning a physical object in a product-lifecycle course in the early 2000s, naming it the Mirrored Spaces Model and describing it as three parts: a physical entity, a digital entity, and the data connection that links their states bidirectionally [13], [4]. The label "digital twin" was attached to this construct around 2011, and NASA gave the first widely cited definition in 2012, casting the twin as an integrated, multi-physics, multi-scale, probabilistic simulation that uses the best available models and sensor history to mirror the life of its flying counterpart [14]. Two features of this lineage matter for what follows. The twin is constituted not by the digital artefact alone but by the coupling of physical and digital together with the data flow between them, and the binding is bidirectional in intent from the outset.

II-B. Two Questions, and an Asset's Logical Counterpart

Agrawal and Fischer reduce the definitional confusion to two questions that must be answered afresh for every application: in a given twin, what exactly is *digital*, and what exactly are we *twinning* [4]? The questions are deceptively simple, and their value is that they force a designer to name the modelled aspects of the asset and the fidelity at which each is represented rather than appealing to the word "twin" as if it carried a fixed meaning. The network-engineering treatment in the Handbook sharpens the vocabulary: a twin couples a *physical object*—a device, product, or resource—with a *logical object* that virtualizes the physical object's relevant characteristics in software, and Minerva and colleagues characterize a twin through properties such as representativeness and contextualization, the requirement that the logical object be credible enough to stand in for the physical one within the context of use [15], [16]. The designer's task is to decide which facets of the asset to represent and how faithfully, not to represent everything.

II-C. The Maturity Ladder: Mirror, Shadow, Twin

The most consequential distinction for a framework is the maturity of the coupling, which the Handbook presents as a progression and which corresponds to the categorical review of Kritzinger and colleagues [11], [17]. At the lowest rung a *digital mirror* is a collection of mathematical models describing the asset and its behaviour but with no live link to it—a simulation that runs independently of any particular physical instance. A *digital shadow* adds a one-way data thread: the digital representation follows the state of its physical counterpart as that state changes, but exercises no influence back on it. Only when a feedback path from the digital side to the physical side is present, whether partial or complete, does the coupling become a *digital twin* in the strict sense, and richer variants layer predictive capability to yield a cognitive twin and human collaboration to yield a collaborative one [11]. Wright and Davidson make the same point from the other direction: the line between a model and a twin is exactly the live, bidirectional binding to a specific physical instance [18]. A framework that aspires to host twins, and not merely models or shadows, must therefore make the feedback path and the distinction between observed and modelled state first-class rather than incidental.

II-D. Dimensions of a Twin

Beyond maturity, the Handbook's deployment methodology decomposes a twin along several further axes that together constitute a design space, and it is this decomposition that we will later read as a specification [11]. The *level* axis locates a twin in a part–whole hierarchy that runs from a component, through equipment composed of components, to a system and finally a system of systems; the same physical artefact may

be modelled as an opaque component in one twin and as a composite with internal structure in another. The *topology* axis describes how the twin is sited relative to its asset: a disconnected or pre-twin used for design-time validation before any link exists, and, once connected, an embedded twin co-located with the asset but constrained in computation, a disjoint twin linked but physically remote, or a joint arrangement in which part of the twin is embedded and part remote. The *synchronization* axis ranges over asynchronous regimes that are event-based, conditional, or on-demand and synchronous regimes that are real-time, near-real-time, or periodic, and the rate may itself vary with the situation. The *decision loop* axis distinguishes an open loop in which a human acts on the twin's output, a closed loop in which the twin acts on the asset directly, and a mixed loop between them. Finally, the *users* axis records that a twin must present suitable interfaces not only to humans but to devices, to software applications, and—significantly—to other twins. This last observation, that a twin is routinely a client of other twins, is what turns a collection of twins into a system and motivates composition as a first-class concern.

II-E. Reference Models, Composition, and a Formal View

These axes sit within the better-known reference models. The Reference Architecture Model Industrie 4.0 organizes assets along hierarchy, lifecycle, and layer axes, and Tao and colleagues extend Grieves' original triad of physical element, virtual element, and data into a five-dimensional model that adds *services* and *connections* as explicit dimensions [11], [2]. The connections dimension—the data and control links among the physical asset, the virtual model, and the services around them—is what the framework must realize as transport, and the services dimension is what it must realize as queryable behaviour. At the level of method, the Handbook situates twin construction within model-based systems engineering and observes that the models composing a complex twin differ both horizontally, by the parts they describe, and vertically, by the concerns or viewpoints they capture, so that building a twin recapitulates the composition of the asset itself and demands principled means of integrating heterogeneous models [19]. Finally, the Handbook offers a formal lens that we will reuse. Sulema and colleagues model a twin as a temporally linked sequence of discrete states, in which the asset occupies exactly one state at each instant and that state is keyed by time and determined by the multimodal data available at that instant [20]. Writing T for an ordered set of time keys and s_t for the state at key $t \in T$, a twin is in this view a time-indexed trajectory

$$\Sigma = \{\, (t,\, s_t) \;:\; t \in T \,\}, \tag{1}$$

together with the machinery that produces, stores, and queries it. We adopt (1) as the abstract object our framework must instantiate per asset, and we will refine s_t into observed and modelled parts in Section IV.

III. Why the Actor Model, and Why the BEAM

The design space of Section II reads, line by line, as a list of properties. We now argue that the actor model supplies those properties and that the BEAM supplies the actor model in a form mature enough to build on.

III-A. A Twin Is an Actor

An actor, in the formulation of Hewitt and the semantics of Agha, is an autonomous entity with private state that interacts with other actors only by asynchronous messages, processing one message at a time and in response updating its state, sending further messages, and creating new actors [5], [6]. Set this beside the twin of Section II. A twin owns the state of one asset and no other; it must not share that state, because representativeness is a property of one physical instance. It evolves in response to inputs—telemetry from its asset, commands from a user, ticks of a clock—which is exactly message handling. It must address and be addressed by other twins, which is the users axis naming "another twin" as a peer. A population of assets becomes a population of actors, their physical couplings become message channels, and the part–whole level axis becomes nesting of actors within actors. The correspondence is not an analogy reached for after the fact; it is the same structure described in two vocabularies. The temporally linked trajectory of (1) is then simply the history of one actor's successive states.

III-B. Why Pure Erlang/OTP, and Not an Actor Library

If the actor model is the right model, one might still implement it as a library on any runtime, and Orleans on the CLR and Akka on the JVM do exactly that [7]. The argument for the BEAM is that it provides as runtime guarantees several things those libraries must approximate, and that the gap matters for twins specifically. BEAM processes are not operating-system threads and not library objects scheduled cooperatively; they are extremely lightweight, preemptively scheduled units of which a single node sustains millions, each with its own heap and its own garbage collection, so that one twin's allocation and collection cannot pause another [10]. This per-process isolation is the runtime expression of the twin's requirement that its state be private and its evolution independent. Failure is contained by the same isolation: a fault in one process does not corrupt another's memory, and the supervisor idiom turns the handling of failure into a declared part of the program structure rather than defensive code scattered through it [9], [12]. Crucially for an unattended twin mirroring a running asset, supervision means that the failure of a twin is followed by its automatic restart into a known state—self-healing is the default rather than an added feature. The BEAM further supports replacing the code of a running system without stopping it, so that the model a twin uses can be updated while the twin continues to track its asset, and it makes message passing between processes location-transparent, so that whether two twins run on one node or on two communicating nodes is a deployment fact and not a difference in the program. Erlang and OTP were built to keep telephone switches running

for years through faults and upgrades, and the properties that purpose demanded—isolation, supervision, live upgrade, distribution—are the properties a long-lived twin of a long-lived asset demands as well [9], [10].

III-C. The Correspondence, Axis by Axis

The fit can be made precise against Section II. The level axis, from component to system of systems, is the supervision hierarchy: a leaf twin is a worker process, an equipment or system twin is a supervisor over its constituent twins, and a system-of-systems twin is a supervisor over those, so that the part–whole structure of the asset and the process tree of the program are the same tree. The maturity ladder is realized by what flows along the edges of that tree and back to the asset: a process that only ingests telemetry is a shadow, and one whose handler emits a command to its asset's adapter closes the loop into a twin. The topology axis is a deployment choice over where processes run, with an embedded twin a process on a constrained node and a disjoint twin a process on a remote node, communicating identically either way because message passing is transparent. The synchronization axis is the process's choice of when to act: a twin that reacts to incoming telemetry messages is event-based, while one that uses the state machine's timeouts to act at fixed intervals is periodic, and a twin can do both. The decision loop axis is whether and how a handler issues actuation, an open loop surfacing output for a human and a closed loop sending a command to the asset. The users axis is the set of senders a twin will accept messages from—human-facing front ends, device adapters, other applications, and peer twins addressed through the registry. Even the heterogeneous, horizontally and vertically differing models of the model-based view [19] have a natural home: distinct concerns can be distinct processes, composed by the same supervision and messaging that compose everything else. None of these mappings requires a mechanism the runtime lacks; each names a use of a mechanism it already has.

IV. Framework Architecture

We now derive the framework. The guiding decision is the division between what the framework owns and what a twin author supplies: everything generic— lifecycle, supervision, addressing, transport, history, and time—is lifted into reusable infrastructure, and the author writes only domain logic against a behaviour. A *behaviour*, in OTP, is an interface together with a generic implementation of the common control flow, so that the author provides a set of callback functions and inherits the rest; the framework is, in essence, a small collection of such behaviours.

IV-A. The `digital_twin` Behaviour

The core of the framework is a behaviour we call `digital_twin`, layered over the OTP `gen_statem` state-machine engine rather than the simpler `gen_server`. The choice is deliberate. Physical assets have operational modes—a pump is stopped, starting, running, or faulted; a valve is open, transitioning, or closed—and a state machine expresses mode-dependent behaviour and mode transitions directly, while its built-in state timeouts give a twin a cost-free notion of staleness: a twin that has heard nothing from its asset within a configured interval transitions itself into a `stale` mode and is no longer mistaken for a live mirror. A purely continuous asset whose state is a single scalar collapses to a one-mode machine, so nothing is lost by defaulting to the more expressive engine and specializing downward. The author implements a small set of callbacks: an `init` that establishes the twin's initial state and configuration, a `handle_observation` invoked when telemetry arrives, a `handle_command` invoked when actuation is requested, a `model_step` that advances the twin's internal model, and a `handle_query` that answers questions about the twin's state. The framework supplies the process, its placement in the supervision tree, its registration for addressing, the wiring to adapters, and the recording of history, so that the author's code concerns only the asset's behaviour.

IV-B. The Observed–Modelled Split and the Divergence

The single design decision that separates this framework's twins from mere shadows is to make the state s_t of (1) carry two components explicitly. The twin holds an *observed* state s_t^{obs}, the most recent telemetry from the asset, and a *modelled* state s_t^{mod}, what the twin's own model predicts should be the case. The `handle_observation` callback updates the former; the `model_step` callback advances the latter according to the asset's dynamics and any commands in effect, so that, writing u_t for the inputs active over the interval ending at t,

$$s_t^{\mathrm{mod}} = f\big(s_{t-1}^{\mathrm{mod}},\, u_t\big)\,, \tag{2}$$

with f the author's `model_step`. The framework then computes the divergence between observation and model under an author-supplied metric d,

$$\delta_t = d\big(s_t^{\mathrm{obs}},\, s_t^{\mathrm{mod}}\big)\,, \tag{3}$$

and exposes it as part of every twin's state. This divergence is what gives a twin a reason to compute rather than merely to store, and it is the quantity in which the phenomena of interest appear: a sensor fault, a modelling error, and a genuine process anomaly all manifest as δ_t crossing a threshold, after which the closed-loop variants may feed the discrepancy back into actuation. In the vocabulary of Section II, (3) operationalizes the line between a shadow and a twin and instruments the maturity ladder directly: a twin that only maintains s_t^{obs} is a shadow, one that also maintains s_t^{mod} and reports δ_t is a twin, and one that acts on δ_t is a closed-loop twin. The construction is faithful to the temporally linked model of (1): each recorded state is keyed by time and now resolves into observed and modelled parts whose relationship is the twin's principal output.

IV-C. Adapters and the Connections Dimension

A twin must not know how its telemetry arrives or how its commands are delivered, because the same asset may be

reached over different transports in different deployments and because a twin should be testable without any transport at all. The framework therefore isolates transport behind a second behaviour, `dt_adapter`, whose implementations translate between the wire—an industrial protocol such as MQTT or OPC UA, a serial link to a microcontroller, or a file—and the twin's observation and command messages. This layer is the connections dimension of the five-dimensional model and the device-communication entity of the manufacturing twin standard rendered as code [2], [21]. The most useful adapter for development is one that drives no hardware at all: a simulator adapter that replays the output of a physics simulation as though it were live telemetry, which lets the entire framework be exercised, and a twin's model validated against ground truth, before any device exists. Whether such an adapter should bridge to a live co-simulation through a port or replay logged simulator output for deterministic, repeatable tests is a genuine choice; the deterministic replay path is the one to build first, because reproducibility is worth more than liveness during development.

IV-D. Composite Twins and the Topology of the Asset

The level axis of Section II becomes architecture through composition. A real asset network—a water distribution system of tanks, pumps, pipes, and junctions—is a graph, and that graph maps onto a supervision hierarchy: leaf twins represent single assets, and a *composite* twin owns a slice of the topology, supervises the twins beneath it, and coordinates them so that, for instance, a pump twin's computed outflow becomes the inflow input of the downstream tank twin at each step. Composites nest, a subsystem twin supervising several composites and a plant twin supervising the subsystems, and this nesting is exactly the aggregate twin of the manufacturing standard [21]. One sub-decision is genuinely open and worth flagging rather than hiding: a composite may step its children explicitly by message, or it may merely supervise while leaf twins exchange values directly along declared edges of the topology. Explicit composite-driven stepping is the more conservative starting point, because it keeps the advance of logical time across the network tractable and centralizes the ordering of updates, whereas peer-to-peer propagation is more decoupled but makes the global time-step harder to reason about. The framework fixes the mechanism—supervision and messaging—and leaves the coordination policy to the composite author.

IV-E. Pluggable Registry, Store, and Clock

Three further concerns are deliberately not fixed, because their right implementation depends on where the framework runs, and each is hidden behind a small behaviour so that the choice is a configuration rather than a rewrite. The *registry* resolves a twin's logical identity to its process; a default backed by the BEAM's built-in process registry suffices on a single node, while a distributed registry such as Horde supports addressing and migration of twins across a cluster [23], [24]. The *store* records the temporally linked history of (1); a bounded in-memory ring buffer is appropriate on a constrained device that cannot retain unbounded history, while a real time-series store is appropriate in the cloud. The *clock* supplies the twin's notion of time; a wall-clock drives live operation, whereas a logical clock drives replay and fast-forward over recorded history, which is what makes the synchronization axis—real-time, periodic, on-demand—a property the operator selects rather than one the framework hard-codes. Pluggability here is not gold-plating; it is the mechanism by which one codebase spans the topology and synchronization axes of the design space.

IV-F. Lifecycle: Resident and Passivated Twins

A final decision concerns whether twins are always-resident processes or are activated on demand and passivated when idle, in the manner of the virtual actors that Orleans introduced and Erleans brought to the BEAM [7], [8]. The two are not in opposition. A twin that mirrors a currently running asset should remain resident, because it must react to telemetry at any moment and because the cost of repeatedly reconstructing its state would be wasteful; a twin of a dormant asset, or a historical twin reconstructed for analysis of a past period, is better passivated, materialized from the store when queried and released when idle. The framework's position is therefore that residency is itself a property of the twin's mode, with live twins resident and dormant or historical twins virtual, and the lifecycle behaviour makes this selectable. This is the point at which the design touches the classic-versus-virtual-actor question directly, and the resolution we adopt is the pragmatic one of letting the asset's liveness, rather than a global policy, decide.

V. The Edge-to-Cloud Continuum

The property that most distinguishes a BEAM-native framework from the alternatives is that it spans the topology axis of Section II in a single runtime and a single codebase, and this deserves separate treatment because it is the crux of the argument.

The BEAM is no longer confined to the server. AtomVM is an implementation of the BEAM small enough to run on microcontrollers such as the ESP32, the GRiSP platform brings Erlang to bare-metal embedded hardware, and the Nerves project packages BEAM applications as the entire software image of an embedded device [25], [26], [27]. Because all of these execute BEAM bytecode and communicate by the same message passing, a twin written against the `digital_twin` behaviour can run as a thin leaf on a sensor-bearing microcontroller, close to the asset and within the embedded topology of the design space, and the very same module can run as a composite in a cloud cluster, with the two communicating as transparently as two processes on one node. The embedded, disjoint, and joint topologies of Section II become, on the BEAM, a question of which node a process is placed on, decided at deployment and not at design. A platform twin cannot offer this, because it assumes the cloud; a ported actor framework cannot offer it, because its runtime does not

descend to the microcontroller. The single-codebase span is therefore not a convenience but the realization of an axis of the standard taxonomy that the alternatives realize only by splitting the system in two.

Two further runtime properties compound the advantage. Live code replacement means a twin's model—the f of (2)—can be improved and loaded into a running twin without stopping it, so the twin need not lose synchrony with an asset that cannot itself be paused, a consideration that is acute precisely for the long-lived infrastructure assets that twins are most valuable for. And supervision means that the failure of an edge twin, a tier where faults are common, is met by automatic restart into a known state rather than by silent divergence. The combination—descend to the device, distribute to the cloud, upgrade in place, recover by default—is available together only because all four are properties of one runtime.

VI. Case Study: A Water-Distribution Twin

We ground the architecture in the domain of water distribution, which exercises every axis of the design space and for which a standard benchmark exists. The C-Town network, derived from a real medium-sized distribution system and disseminated through the Battle of the Attack Detection Algorithms, comprises tanks, pumps, valves, and pipes organized into district metered areas, and has become a common substrate for studies of monitoring and anomaly detection [30]. Its dynamics can be generated by established tools—EPANET for hydraulic simulation and the Water Network Tool for Resilience built upon it—which makes it an ideal source of ground-truth telemetry for a simulator adapter [31], [32].

The canonical instance we use to exercise the framework is the smallest composite that is not trivial: a pump feeding a tank. The pump twin and the tank twin are leaf processes under a composite that owns this fragment of topology. At each logical step the composite advances its children, the pump twin's `model_step` computes an outflow from its mode and setpoint, and that outflow is delivered as the inflow input to the tank twin, whose own `model_step` integrates it against demand to update the predicted level, realizing (2) across two coupled twins. The simulator adapter replays EPANET-generated telemetry for the same fragment as observations, so each twin maintains the observed level alongside its modelled level and reports the divergence of (3); a model-in-the-loop check then reduces to asking whether each twin's modelled trajectory tracks the simulator's ground-truth trajectory within tolerance, and an injected discrepancy—a simulated leak absent from the model—appears as a growing δ_t that a closed-loop variant could act upon. Because the adapter drives the system from logged simulation rather than hardware, the entire case study is deterministic and repeatable, and because the twins are ordinary supervised processes, the failure of the pump twin mid-run is followed by its restart and re-registration without disturbing the tank twin. The design described here is consistent with, and motivated by, a working Erlang prototype of a twin-per-asset system over the same benchmark; the present paper abstracts that prototype into the framework whose generality it was built to demonstrate.

VII. Related Work

The framework sits at the intersection of three bodies of work. The first is the DT platform. Azure Digital Twins offers a managed graph of twin instances described by a modelling language, and Eclipse Ditto provides an open abstraction for device twins with state synchronization and a query interface [28], [29]. These are mature and operationally capable, but they are cloud-resident and treat the twin as a managed object rather than a live computational actor, and neither descends to the embedded topology that the design space includes. The second body of work is the actor and virtual-actor runtime. Orleans established the virtual-actor abstraction and demonstrated it at scale, Akka brought actors to the JVM, and Dapr generalized the pattern as a sidecar; on the BEAM, Erleans ports the virtual-actor idea, Horde provides distributed process registration and supervision, and Partisan re-engineers the distribution layer for larger clusters [7], [8], [23], [24]. Our contribution relative to this lineage is not a new actor mechanism but the observation, developed in Section III, that the twin's requirements align with the actor model so closely that the native BEAM mechanisms suffice, together with the edge-to-cloud span that the ported frameworks do not provide. The third body of work is the standards and conceptual literature, including the manufacturing twin framework of ISO 23247, the concepts and terminology of ISO/IEC 30173, and the surveys and conceptual models that the Handbook collects [21], [22], [1], [3], [16]; we use this literature as the specification against which the framework is designed rather than as a competitor. To our knowledge, a digital-twin framework conceived as a direct realization of this design space in pure Erlang/OTP, and spanning the edge-to-cloud continuum on a single runtime, has not previously been articulated.

VIII. Discussion and Limitations

The argument of this paper is that a runtime can fit a problem structurally, and that the BEAM fits the DT problem because the actor model the runtime implements natively is the model the twin demands. Honesty requires marking where the fit is partial. The clearest limitation is numerical: the BEAM is optimized for concurrency, message passing, and fault tolerance, not for the dense floating-point computation that high-fidelity physical models, finite element analyses, and large-scale optimization require, and a twin whose model is computationally heavy will need to delegate that computation through native implemented functions or to an external solver rather than express it in Erlang. This is less a refutation than a boundary: the framework's contribution is the orchestration, lifecycle, and composition of twins, and heavy numerics belong behind the `model_step` callback as a called service. A second limitation is ecosystem maturity. The platforms enjoy extensive tooling for modelling languages, visualization, and integration that a research framework does not, and although

the BEAM's primitives are mature, twin-specific libraries on top of them are not, so early adopters carry more of the burden themselves. A third concerns the open design questions named in Section IV: whether composites step children explicitly or leaves propagate peer-to-peer, whether the simulator adapter co-simulates live or replays logs, and what policy governs passivation are decisions whose best answers likely depend on the domain and which a single framework should parameterize rather than legislate.

Several directions follow. The temporally linked formalism of (1) and the message protocols among composite and leaf twins are amenable to formal specification and verification, which the BEAM community has pursued for protocol-bearing systems and which would let one prove properties of a twin network rather than test for them. The edge claim of Section V invites empirical substantiation: a quantitative study of a twin running under AtomVM on representative microcontroller hardware, measuring the footprint and latency of the `digital_twin` behaviour at the edge, would convert the architectural argument into measured evidence. And because twins of infrastructure are attractive targets, the security of the twin network—the authentication of adapters, the integrity of the observation and command channels, and the containment of a compromised twin by the same isolation that contains a failed one—is a natural and pressing extension of the present work.

IX. Conclusion

The digital twin's definitional fragmentation has obscured a structural fact: the design space the field has converged on, as codified in the *Handbook of Digital Twins*, is a description of autonomous, stateful, hierarchically composed, fault-prone, long-lived, and variably connected entities, and that is a description of actors. We have argued that the Erlang/OTP runtime, the BEAM, supplies the actor model not as a library but as its native execution model, and that each axis of the DT design space—level, maturity, topology, synchronization, decision loop, and users—maps onto a mechanism the runtime already provides. From that correspondence we derived a framework in pure Erlang whose twins are supervised state machines carrying an explicit observed-versus-modelled split, whose divergence operationalizes the shadow-to-twin maturity ladder, whose asset topology is a supervision hierarchy, and whose transport, addressing, history, and time are pluggable behaviours; and we identified the edge-to-cloud span, in which one twin module runs from microcontroller to cluster with live upgrade and default recovery, as the property that distinguishes a BEAM-native framework from both platforms and ported actor frameworks. Building a twin framework on the BEAM is, in the end, less an act of construction than of recognition, because the hardest requirements are already mechanisms in the runtime. The water-distribution case study shows the recognition is workable; the limitations show where it must borrow; and the structural fit suggests the recognition is worth pursuing.

References

[1] Z. Lyu, Ed., *Handbook of Digital Twins*. Boca Raton, FL, USA: CRC Press, Taylor & Francis, 2024.

[2] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, 2019.

[3] A. Fuller, Z. Fan, C. Day, and C. Barlow, "Digital twin: Enabling technologies, challenges and open research," *IEEE Access*, vol. 8, pp. 108952–108971, 2020.

[4] A. Agrawal and M. Fischer, "What is digital and what are we twinning?: A conceptual model to make sense of digital twins," in *Handbook of Digital Twins*, Z. Lyu, Ed. Boca Raton, FL, USA: CRC Press, 2024, ch. 2, pp. 13–29.

[5] C. Hewitt, P. Bishop, and R. Steiger, "A universal modular ACTOR formalism for artificial intelligence," in *Proc. 3rd Int. Joint Conf. Artif. Intell. (IJCAI)*, 1973, pp. 235–245.

[6] G. Agha, *Actors: A Model of Concurrent Computation in Distributed Systems*. Cambridge, MA, USA: MIT Press, 1986.

[7] P. A. Bernstein, S. Bykov, A. Geller, G. Kliot, and J. Thelin, "Orleans: Distributed virtual actors for programmability and scalability," Microsoft Research, Redmond, WA, USA, Tech. Rep. MSR-TR-2014-41, 2014.

[8] T. Sloughter *et al.*, "Erleans: Erlang Orleans-style virtual actors," Erleans Project. [Online]. Available: https://github.com/erleans/erleans

[9] J. Armstrong, "Making reliable distributed systems in the presence of software errors," Ph.D. dissertation, Dept. Microelectron. Inf. Technol., Royal Inst. Technol. (KTH), Stockholm, Sweden, 2003.

[10] F. Cesarini and S. Vinoski, *Designing for Scalability with Erlang/OTP: Implementing Robust, Fault-Tolerant Systems*. Sebastopol, CA, USA: O'Reilly Media, 2016.

[11] M. A. Hamzaoui and N. Julien, "A generic deployment methodology for digital twins – First building blocks," in *Handbook of Digital Twins*, Z. Lyu, Ed. Boca Raton, FL, USA: CRC Press, 2024, ch. 7, pp. 102–121.

[12] Ericsson AB, "OTP design principles," Erlang/OTP System Documentation. [Online]. Available: https://www.erlang.org/doc/design_principles/des_princ.html

[13] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems*, F.-J. Kahlen, S. Flumerfelt, and A. Alves, Eds. Cham, Switzerland: Springer, 2017, pp. 85–113.

[14] E. H. Glaessgen and D. S. Stargel, "The digital twin paradigm for future NASA and U.S. Air Force vehicles," in *Proc. 53rd AIAA/ASME/ASCE/AHS/ASC Struct., Struct. Dyn. Mater. Conf.*, Honolulu, HI, USA, 2012, AIAA 2012-1818.

[15] P. H. Diniz, C. E. Rothenberg, and J. F. de Rezende, "When digital twin meets network engineering and operations," in *Handbook of Digital Twins*, Z. Lyu, Ed. Boca Raton, FL, USA: CRC Press, 2024, ch. 3, pp. 30–50.

[16] R. Minerva, G. M. Lee, and N. Crespi, "Digital twin in the IoT context: A survey on technical features, scenarios, and architectural models," *Proc. IEEE*, vol. 108, no. 10, pp. 1785–1824, 2020.

[17] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, "Digital twin in manufacturing: A categorical literature review and classification," *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.

[18] L. Wright and S. Davidson, "How to tell the difference between a model and a digital twin," *Adv. Model. Simul. Eng. Sci.*, vol. 7, art. 13, 2020.

[19] S. P. Kovalyov, "Key technologies of digital twins: A model-based perspective," in *Handbook of Digital Twins*, Z. Lyu, Ed. Boca Raton, FL, USA: CRC Press, 2024, ch. 6, pp. 85–101.

[20] Y. Sulema, A. Pester, I. Dychka, and O. Sulema, "Digital twin model formal specification and software design," in *Handbook of Digital Twins*, Z. Lyu, Ed. Boca Raton, FL, USA: CRC Press, 2024, ch. 12, pp. 203–220.

[21] *Automation Systems and Integration – Digital Twin Framework for Manufacturing – Part 1: Overview and General Principles*, ISO Standard 23247-1:2021, International Organization for Standardization, Geneva, Switzerland, 2021.

[22] *Digital Twin – Concepts and Terminology*, ISO/IEC Standard 30173:2023, International Organization for Standardization, Geneva, Switzerland, 2023.

[23] M. Schaefermeier *et al.*, "Horde: Distributed, fault-tolerant process registry and dynamic supervisor for Elixir," Horde Project. [Online]. Available: https://github.com/derekkraan/horde

[24] C. Meiklejohn, H. Miller, and P. Alvaro, "Partisan: Scaling the distributed actor runtime," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2019, pp. 63–76.
[25] D. Mazza *et al.*, "AtomVM: The Erlang virtual machine for IoT devices," AtomVM Project. [Online]. Available: https://www.atomvm.net/
[26] Stritzinger GmbH, "GRiSP: Bare-metal Erlang for embedded systems." [Online]. Available: https://www.grisp.org/
[27] Nerves Project Authors, "Nerves: Craft and deploy bulletproof embedded software in Elixir." [Online]. Available: https://nerves-project.org/
[28] Microsoft, "Azure Digital Twins documentation." [Online]. Available: https://learn.microsoft.com/azure/digital-twins/
[29] Eclipse Foundation, "Eclipse Ditto: Digital twins for IoT." [Online]. Available: https://eclipse.dev/ditto/
[30] R. Taormina *et al.*, "Battle of the attack detection algorithms: Disclosing cyber attacks on water distribution networks," *J. Water Resour. Plan. Manage.*, vol. 144, no. 8, art. 04018048, 2018.
[31] L. A. Rossman, *EPANET 2 Users Manual*, U.S. Environmental Protection Agency, Cincinnati, OH, USA, Rep. EPA/600/R-00/057, 2000.
[32] K. A. Klise, M. Bynum, D. Moriarty, and R. Murray, "A software framework for assessing the resilience of drinking water systems to disasters with an example earthquake case study," *Environ. Model. Softw.*, vol. 95, pp. 420–431, 2017.